Ejercicio genérico de arquitectura de sistemas distribuidos

José Manuel Aroca Fernández

Índice de contenidos:

1. Que poner en una arquitectura: 4
2. OSS y BSS 4
   OSS - Sistemas de Soporte a las Operaciones 4
   BSS - Sistemas de Soporte al Negocio 4
   Cómo aplicar OSS y BSS a Arquitecturas de Sistemas 4
   Diferencias Clave entre OSS y BSS 5
   Beneficios de una buena arquitectura OSS/BSS 5
3. Estructura de una documento de arquitectura 6
4. Sistemas empresariales comunes (definiciones clave) 12
   ERP (Enterprise Resource Planning) 12
   CRM (Customer Relationship Management) 12
   WMS (Warehouse Management System) 13
   TMS (Transportation Management System) 13
   OMS (Order Management System) 13
   CMS (Content Management System) 14
   IAM (Identity and Access Management) 14
   BI / Data Warehouse 14
   Payment Gateway 15
   Marketplace 15
5. Componentes técnicos frecuentes en arquitectura 15
   API Gateway 15
   Message Broker 15
   Cache 16
   Search Engine 16
   CDN 16
6. Cómo definir requisitos funcionales 16
7. Requisitos no funcionales 17
8. Tips para diseñar arquitectura 17
   Identifica bounded contexts 18
   Identifica integraciones externas 18
   Detecta cuellos de botella 18
   Define datos maestros 18
9. Framework mental para entrevistas o ejercicios 18
10. Pasos a seguir 19
11. Enunciado ejercicio 20

1. Que poner en una arquitectura:

Cuando diseñas arquitecturas de sistemas (por ejemplo para e-commerce, logística, banca, retail, etc.), conviene manejar un vocabulario claro de **tipos de sistemas**, **componentes empresariales** y **criterios de requisitos**. Esto evita diseños demasiado técnicos sin entender primero el negocio.

2. OSS y BSS

En arquitecturas empresariales, especialmente en **telecom**, utilities, banca y grandes operaciones digitales, se usan mucho los términos **BSS** y **OSS** para separar sistemas orientados al negocio vs operación técnica.

OSS - Sistemas de Soporte a las Operaciones

El OSS es el conjunto de herramientas que permiten a los ingenieros y técnicos gestionar la red.

- **Gestión de Red:**Supervisa la infraestructura física y lógica.
- **Provisión de Servicios (Provisioning):** Configura los servicios en la red para un cliente específico.
- **Garantía de Servicio (Assurance):** Monitorea el rendimiento y soluciona fallos para cumplir con los Acuerdos de Nivel de Servicio (SLA).
- **Inventario de Red:** Mantiene un registro de los recursos físicos (routers, cables) y lógicos (direcciones IP)

BSS - Sistemas de Soporte al Negocio

El BSS gestiona la interacción con el cliente y las operaciones financieras.

- **CRM (Customer Relationship Management):**Gestiona los datos del cliente, interacciones y soporte técnico.
- **Gestión de Pedidos (Order Management):**Procesa las solicitudes de compra de servicios.
- **Facturación y Cobro (Billing & Charging):**Calcula los cargos y genera las facturas.
- **Gestión de Productos:**Define los paquetes de servicios y precios.

Cómo aplicar OSS y BSS a Arquitecturas de Sistemas

La integración de OSS y BSS es fundamental para crear una experiencia de cliente fluida y una red eficiente. Su aplicación moderna se centra en la automatización y la nube.

A. Enfoque en Microservicios y API-First

- **Desacoplamiento:** En lugar de sistemas monolíticos, se aplican arquitecturas basadas en microservicios, donde cada función (ej. "facturación" o "gestión de inventario") funciona de forma independiente.

- **Integración API:** Los sistemas BSS se comunican con los OSS en tiempo real a través de APIs abiertas , permitiendo que la compra de un servicio (BSS) active inmediatamente la red (OSS).

B. Automatización y Autoprovisión

- **Escenario:** Un cliente compra un paquete de internet por fibra en un portal web (BSS).
- **Aplicación:** El sistema BSS envía una orden automática al OSS, que configura el puerto del router (provisión) y activa el servicio sin intervención humana.

C. Integración en la Nube (Cloud-Native)

- Se migran los componentes OSS/BSS a plataformas de nube (como Red Hat, AWS, Azure) para garantizar escalabilidad bajo demanda, especialmente para redes 5G y servicios IoT.

D. Fusión con SIG (Sistemas de Información Geográfica)

- **Uso:**Para operadores de red (fibra/telco), se integra el inventario de red (OSS) y los datos de clientes (BSS) con herramientas como ArcGIS de Esri, permitiendo visualizar dónde hay cobertura para vender y dónde hay fallos geolocalizados.

Diferencias Clave entre OSS y BSS

Característica	OSS (Operaciones)	BSS (Negocio)
Foco principal	Infraestructura y Red	Cliente ,Facturación y otras funcionalidades.
Usuario clave	Ingenieros / Técnicos	Ventas / Agentes de Atención/Clientes
Objetivo	Alta disponibilidad de red	Satisfacción del cliente / Ingresos
Ejemplo	Configuración de Router / Inventario /Interfaz con operador logístico / Interfaz con la pasarela de pagos	Factura mensual / CRM

Beneficios de una buena arquitectura OSS/BSS

- **Reducción del tiempo de provisión (Time-to-market):** Los servicios se activan más rápido.

- **Mejor experiencia de cliente:** Facturación exacta y soporte técnico más rápido.
- **Optimización de costos:** Automatización de tareas repetitivas y mejor uso de recursos de red.

3. Estructura de una documento de arquitectura

Una **estructura estándar de un documento de arquitectura de sistemas** suele seguir plantillas como **IEEE 42010**, **arc42**, TOGAF o formatos internos de empresas. En entrevistas, consultoría o proyectos reales, normalmente se usa una versión simplificada pero muy clara.

A modo de resumen los puntos son:

1. Contexto
2. Requisitos
3. Arquitectura alto nivel
4. Componentes
5. Datos
6. Integraciones
7. Seguridad
8. Escalabilidad
9. Riesgos

Si miramos en detalle:

---

1. Portada

- Nombre del sistema
- Versión del documento
- Autor
- Fecha
- Estado (draft/final/review)

---

2. Resumen ejecutivo

Explica brevemente:

- objetivo del sistema
- problema de negocio

- alcance
- stakeholders principales

Ejemplo:

Plataforma e-commerce para gestionar ventas online, integración con proveedores logísticos y múltiples pasarelas de pago.

---

3. Objetivos de negocio

Aquí defines el **por qué** del sistema.

Ejemplo:

- aumentar ventas online
- reducir tiempos logísticos
- integrar proveedores externos
- mejorar experiencia cliente

---

4. Alcance funcional

Qué incluye:

- gestión de usuarios
- catálogo
- pedidos
- pagos
- logística

Qué no incluye:

- contabilidad interna
- RRHH

---

5. Stakeholders

- cliente final
- administradores
- proveedores
- equipo técnico
- partners externos

---

6. Requisitos funcionales

Ejemplo:

- registro/login
- búsqueda productos
- checkout
- devoluciones

---

7. Requisitos no funcionales

Muy importante:

- disponibilidad
- seguridad
- rendimiento
- escalabilidad
- compliance
- observabilidad

Ejemplo:

El sistema soportará 50.000 usuarios concurrentes.

---

8. Arquitectura de alto nivel

Diagrama general:

Aquíí en función del tipo de sistema , y del los conocimientos técnicos del cliente usaremos una o varias de las representaciones vistas:

- Esquema
- 3 capas
- 4 capas
- 7 capas
- Otros intermedios siempre que estén justificados y se ciñan a las reglas de este documento.

Ejemplo componentes:

- web
- mobile
- API gateway

- microservicios
- ERP
- CRM
- WMS

Herramientas típicas:

- Draw.io
- Lucidchart
- Microsoft Visio

---

9. Arquitectura lógica / módulos

Separación por dominios:

- usuarios
- pagos
- pedidos
- inventario
- logística

Muy relacionado con:

Domain-Driven Design

---

10. Arquitectura física / infraestructura

Dónde corre:

- cloud
- on-premise
- híbrido

Ejemplo:

- Amazon Web Services
- Microsoft Azure
- Google Cloud

Incluye:

- servidores
- balanceadores

- redes
- contenedores

---

11. Modelo de datos

Entidades principales:

- usuarios
- pedidos
- productos
- pagos

Bases de datos:

- SQL
- NoSQL
- Data warehouse

Ejemplos:

- PostgreSQL
- MongoDB
- Redis

---

12. Integraciones externas

Muy importante en sistemas reales:

- ERP
- CRM
- proveedores logísticos
- pagos
- marketplaces

Ejemplo:

- Stripe
- PayPal
- DHL

---

13. Seguridad

- autenticación
- autorización
- cifrado
- auditoría

Herramientas:

- Keycloak
- OAuth2
- JWT

---

14. DevOps / despliegue

- CI/CD
- entornos
- monitorización

Herramientas:

- Jenkins
- GitLab
- Docker
- Kubernetes

---

15. Riesgos técnicos

Ejemplo:

- dependencia de terceros
- cuellos de botella
- deuda técnica

---

16. Costes estimados

- infraestructura
- licencias
- mantenimiento
- personal

---

17. Roadmap / fases

Fase 1 → MVP
Fase 2 → integraciones
Fase 3 → escalado

---

18. Anexos

- diagramas detallados
- glosario
- APIs
- contratos técnicos

4. Sistemas empresariales comunes (definiciones clave)

ERP (Enterprise Resource Planning)

Sistema que integra procesos internos de la empresa:

- compras
- inventario
- facturación
- contabilidad
- RRHH
- producción
- proveedores

Ejemplos:

- SAP ERP
- Oracle ERP Cloud
- Odoo
- Microsoft Dynamics 365

CRM (Customer Relationship Management)

Gestiona la relación con clientes:

- leads
- oportunidades

- campañas
- ventas
- atención al cliente
- historial comercial

Ejemplos:

- Salesforce
- HubSpot
- Microsoft Dynamics 365

WMS (Warehouse Management System)

Gestión de almacenes:

- stock
- ubicación
- picking
- packing
- inventarios

Ejemplo:

- Manhattan Associates WMS

TMS (Transportation Management System)

Gestión logística y transporte:

- rutas
- transportistas
- tracking
- entregas

Ejemplo:

- DHL integraciones frecuentes

OMS (Order Management System)

Gestiona pedidos:

- creación

- pagos
- estados
- devoluciones
- cancelaciones

Muy usado en e-commerce.

CMS (Content Management System)

Gestiona contenido web:

- productos
- banners
- páginas

Ejemplos:

- WordPress
- Adobe Experience Manager

IAM (Identity and Access Management)

Autenticación/autorización:

- login
- roles
- permisos
- SSO

Ejemplo:

- Keycloak

BI / Data Warehouse

Análisis y reporting:

- dashboards
- KPIs
- predicciones

Ejemplos:

- Power BI

- Tableau

Payment Gateway

Pasarela de pago:

- tarjeta
- PayPal
- Bizum
- transferencias

Ejemplos:

- Stripe
- PayPal

Marketplace

Plataforma con múltiples vendedores

Ejemplo:

- Amazon
- eBay

---

5. Componentes técnicos frecuentes en arquitectura

API Gateway

Entrada única a microservicios

Ejemplo:

- Kong
- Spring Cloud Gateway

Message Broker

Comunicación asíncrona

Ejemplos:

- RabbitMQ
- Apache Kafka

Cache

Ejemplo:

- Redis

Search Engine

Ejemplo:

- Elasticsearch

CDN

Ejemplo:

- Cloudflare

---

6. Cómo definir requisitos funcionales

Preguntas clave:

Usuarios

- ¿Quién usa el sistema?
- cliente
- admin
- proveedor
- operador logístico

Procesos

- ¿Qué flujo ejecutan?

Ejemplo e-commerce:

- registro
- compra
- pago
- envío
- devolución

Integraciones

- bancos
- operadores logísticos
- proveedores
- sistemas legacy
- marketplaces externos

Datos

- productos
- pedidos
- clientes
- facturas

---

7. Requisitos no funcionales

Muy olvidados en entrevistas/diseño:

- escalabilidad
- seguridad
- disponibilidad
- rendimiento
- auditoría
- backup
- compliance (GDPR, PCI)

Ejemplo:

soportar 100k usuarios concurrentes

---

8. Tips para diseñar arquitectura

Empieza por negocio, no por tecnología

Error típico:

“voy a usar microservicios”

Primero entender procesos.

Identifica bounded contexts

Ejemplo:

- pagos
- catálogo
- logística
- clientes

Relacionado con Domain-Driven Design

Identifica integraciones externas

Muy común olvidarlas:

- pagos
- email
- SMS
- shipping

Detecta cuellos de botella

Ejemplo:

- checkout
- búsquedas
- stock

Define datos maestros

- producto
- cliente
- inventario

---

9. Framework mental para entrevistas o ejercicios

Usa este orden:

1. Objetivo de negocio
2. Usuarios
3. Casos de uso
4. Datos

5. Integraciones
6. Escalabilidad
7. Seguridad
8. Arquitectura técnica

10. Pasos a seguir

Pasos para el diseño de la arquitectura de un sistema

1.Entender los requisitos

1. Recopila requisitos funcionales (qué debe hacer el sistema) y no funcionales (rendimiento, seguridad, escalabilidad, disponibilidad, etc.).
2. Identifica restricciones técnicas, de negocio y regulatorias.

2.Analizar el contexto y los actores

1. Define los usuarios, sistemas externos y flujos de interacción.
2. Elabora diagramas de contexto y casos de uso.

3.Definir la visión y los objetivos arquitectónicos

1. Establece los principios y prioridades (por ejemplo, modularidad, mantenibilidad, time-to-market).
2. Determina los atributos de calidad clave (por ejemplo, escalabilidad, resiliencia, seguridad).

4.Dividir el sistema en módulos o componentes

1. Identifica los dominios o bounded contexts.
2. Decide si usarás monolito, microservicios, arquitectura hexagonal, orientada a eventos, etc.
3. Define responsabilidades y límites claros para cada componente.

Pasos para el diseño de la arquitectura de un sistema

5. Diseñar la comunicación y la integración

1. Elige protocolos y patrones de integración (REST, eventos, colas, gRPC, etc.).
2. Define contratos y APIs entre componentes.

6.Seleccionar tecnologías y herramientas

1. Evalúa frameworks, lenguajes, bases de datos, servicios cloud, herramientas de CI/CD, etc.
2. Considera compatibilidad, soporte y comunidad.

7.Diseñar la seguridad y la gestión de datos

1. Define mecanismos de autenticación, autorización y auditoría.
2. Establece políticas de protección de datos, backup y recuperación.

8.Planificar la escalabilidad y la resiliencia

1. Diseña para tolerancia a fallos, balanceo de carga, replicación y recuperación ante desastres.
2. Considera patrones como Circuit Breaker, Retry, Bulkhead.

9.Documentar la arquitectura

1. Crea diagramas (componentes, despliegue, secuencia, flujo de datos).
2. Documenta decisiones arquitectónicas y trade-offs.

10.Validar y revisar la arquitectura

1. Realiza revisiones con stakeholders y expertos.
2. Haz prototipos o pruebas de concepto para riesgos críticos.
3. Ajusta la arquitectura según feedback y nuevas necesidades.

11. Enunciado ejercicio

Ejercicio de Arquitectura de Sistemas: Ecosistema Digital de El Corte Inglés

Objetivo: Diseñar la arquitectura funcional y técnica de un ecosistema omnicanal similar al de El Corte Inglés, integrando retail, supermercado, pagos, logística, fidelización y viajes.

Contexto funcional

La empresa dispone de una aplicación móvil principal que integra múltiples líneas de negocio: ecommerce retail, supermercado, pagos financieros, reservas de viajes, ticket digital y programas de fidelización.

Módulos funcionales requeridos

Módulo	Descripción
Retail	Catálogo, carrito, checkout, pedidos
Supermercado	Compra recurrente y entregas
Pagos	Tarjeta digital y financiación
Logística	Entrega y recogida en tienda
Fidelización	Cupones y promociones
Viajes	Reservas e itinerarios
Ticket Digital	Historial de compras y devoluciones

Ejercicio práctico

1. Diseñar arquitectura funcional de alto nivel.
2. Identificar microservicios necesarios.
3. Diseñar base de datos principal.
4. Definir APIs entre sistemas.
5. Diseñar integración móvil-web-backend.
6. Proponer despliegue en cloud.
7. Diseñar escalabilidad para Black Friday.

Arquitectura esperada (referencia)

Frontend Mobile/Web → API Gateway → Microservices → ERP/CRM/OMS/WMS → Base de datos → Analytics

Tecnologías sugeridas

• Frontend: React / Flutter
• Backend: Python / Java / Spring Boot
• ERP: Odoo / SAP
• Containers: Docker
• Orquestación: Kubernetes
• Cloud: AWS / Azure / Google Cloud
• Mensajería: RabbitMQ / Kafka

Entregable del alumno: diagrama de arquitectura + explicación técnica + propuesta de escalabilidad.

1. App principal de El Corte Inglés

Es la aplicación central del ecosistema.

Funcionalidades principales

Catálogo unificado

Permite comprar productos de múltiples categorías:

- moda
- electrónica
- hogar
- deportes
- libros
- perfumería
- alimentación
- juguetes
- bricolaje

También integra marcas del grupo:

- Hipercor
- Supercor
- Sfera
- Primeriti
- Motortown
- Bricor

Flujo:

Usuario → búsqueda → catálogo → carrito → pago → entrega

---

2. Gestión de pedidos

La app permite múltiples opciones logísticas:

Entrega rápida

- entrega en 2 horas
- entrega en el día
- entrega programada

Entrega estándar

- 48 horas

Recogida física

- click & collect
- recogida en tienda
- Click & Car

Flujo:

Pedido → preparación → logística → entrega/recogida

---

3. Modo tienda (integración física)

Una funcionalidad muy interesante.

Permite:

- localizar tiendas cercanas
- consultar departamentos
- localizar productos
- consultar eventos
- navegación dentro del centro

Flujo:

Cliente → geolocalización → tienda → departamentos → compra

---

4. Pago móvil

Integra la Financiera El Corte Inglés.

Funciones:

- pagar con móvil
- usar tarjeta digital
- autenticación biométrica
- consulta de movimientos
- financiación

Flujo:

Compra → autenticación → pago → confirmación

---

5. Ticket digital

Digitaliza postventa:

- elimina tickets físicos
- guarda historial
- facilita devoluciones
- seguimiento de compras

Flujo:

Compra → ticket digital → historial → devolución

---

6. Parking inteligente

Muy integrado con centros físicos.

Permite:

- reconocer matrícula
- apertura automática de barreras
- pago automático

Flujo:

Vehículo → entrada parking → compra → salida automática

---

7. Promociones y fidelización

La app gestiona:

- cupones
- descuentos
- promociones personalizadas
- campañas

Flujo:

CRM → segmentación → oferta → compra

---

8. Supermercado

Hipercor y supermercado digital:

- compra online
- carrito recurrente

- entrega a domicilio
- categorías por lineales virtuales

Flujo:

Cliente → supermercado → carrito → entrega

---

9. Viajes

Viajes El Corte Inglés tiene funcionalidades propias:

- reservas
- eventos
- viajes corporativos
- itinerarios
- documentación

Flujo:

Reserva → pago → documentación → viaje

Sistemas backend implicados

Probablemente integran:

- ERP
- CRM
- OMS (Order Management System)
- WMS (Warehouse Management System)
- pasarelas de pago
- APIs externas
- logística
- analítica